

Mini Project Report
on
Smart Grievance Handling System

Under the Course
“ Database Management System Laboratory”

Submitted By

Mr. Atharva Deshmukh [124B1F150]
Mr. Maahit Bhawsar [124B1F154]
Mr. Vijay Jathare [125B2F003]
Mr. Sahil Dhumal [125B2F006]

**Under the guidance of
Mrs. Alpana Borse**



Department of Information Technology Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

Academic Year 2025-26

Abstract:

The **Smart Grievance Handling System** is a digitised civic grievance platform designed to bridge the gap between citizens and municipal authorities through automated reporting and resolution tracking. The project followed a full-stack **methodology**, utilizing a **React.js** frontend, **Node.js** backend, and a **MySQL** database equipped with automated triggers for priority logging. Key **objectives** included establishing secure **JWT-based** access and a real-time activity feed for transparency. The primary **outcomes** were a functional automated scoring engine and a comprehensive admin portal that allows for prioritized status management and citizen feedback collection.

Introduction:

In rapidly growing urban environments, citizens frequently encounter civic problems such as broken roads, water supply failures, uncleared garbage, power outages, and damaged public parks. Traditionally, citizens report these issues through phone calls, physical visits to municipal offices, or informal social media posts channels that are disorganised, difficult to track, and prone to information loss. This creates a trust deficit between citizens and local authorities and results in delayed or unresolved civic problems.

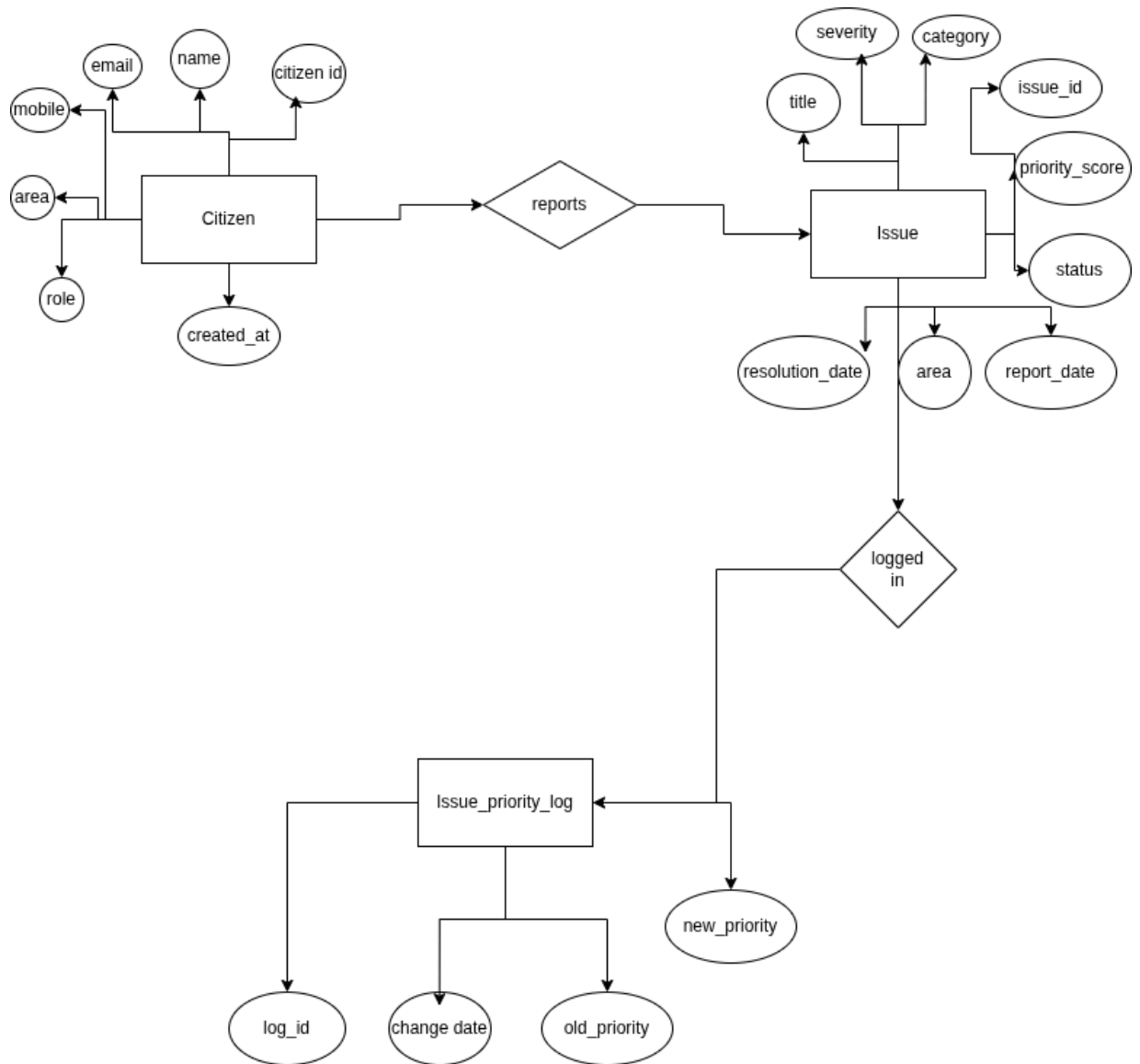
The Smart Grievance Handling System addresses this challenge by providing a centralized digital platform where citizens can formally register complaints, receive a unique issue identifier, and monitor real-time status updates as authorities act on each report. The system also exposes a public-facing live feed of resolved issues and categorical statistics, promoting transparency and accountability. The entire workflow from complaint submission to resolution and citizen feedback is managed through a relational database governed by triggers, constraints, and role-based access control.

Objectives:

- To design and implement a normalised relational database system for managing citizens, civic issues, and audit logs using MySQL.
- To implement SQL queries, constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, ENUM), and database triggers for automated priority score computation and logging.
- To develop a RESTful backend API using Node.js and Express.js with JWT authentication and encrypt password security
- To build a responsive React.js frontend with role-based views for Citizens and Administrators.
- To provide real-time public statistics (resolved count, active issues, average response time, citizen rating) and a live activity feed on the home page.
- To enable citizens to update issue severity, track resolution status, and submit star-rating feedback upon resolution.

Database Design:

- ER Diagram



- **Tables and Attributes**

Table 1: Citizen:

Column Name	Type	Constraint	Description
citizen_id	INT	PK, AUTO_INC	Unique citizen identifier
name	VARCHAR(255)	NOT NULL	Full name of the citizen
email	VARCHAR(255)	UNIQUE, NOT NULL	Registered email address
mobile	VARCHAR(20)	NOT NULL	Contact mobile number
area	VARCHAR(255)	NOT NULL	Citizen's residential area/ward
password	VARCHAR(255)	NOT NULL	Encrypt-hashed password
role	ENUM	DEFAULT citizen	citizen or admin
created_at	TIMESTAMP	DEFAULT NOW()	Registration timestamp

Table 2: Issue:

Column Name	Data Type	Constraint	Description
issue_id	INT	PK, AUTO_INC	Unique issue identifier
title	VARCHAR(255)	NOT NULL	Short title of the issue
description	TEXT	NOT NULL	Detailed description
area	VARCHAR(255)	NOT NULL	Location of the issue
category	ENUM	NOT NULL	Roads/Water/Sanitation/Electricity/Parks/Others
severity	ENUM	NOT NULL	Low / Medium / High
priority_score	INT	DEFAULT 0	Auto-computed: Low=30, Medium=60, High=90
status	ENUM	DEFAULT Open	Open / In Progress / Resolved
report_date	TIMESTAMP	DEFAULT NOW()	Date & time of submission
resolution_date	TIMESTAMP	NULL	Date & time of resolution
citizen_id	INT	FK → Citizen	Reporting citizen reference

Table 3: Issue_Priority_Log

Column Name	Data Type	Constraint	Description
log_id	INT	PK, AUTO_INC	Unique log entry identifier
issue_id	INT	FK → Issue	Issue being logged
old_priority	INT	NULL	Priority score before change
new_priority	INT	NULL	Priority score after change
change_date	TIMESTAMP	DEFAULT NOW()	Timestamp of the log entry

- Primary Keys and Foreign Keys
 1. Citizen.citizen_id — PRIMARY KEY
 2. Issue.issue_id — PRIMARY KEY
 3. Issue.citizen_id — FOREIGN KEY REFERENCES Citizen(citizen_id) ON DELETE CASCADE
 4. Issue_Priority_Log.log_id — PRIMARY KEY
 5. Issue_Priority_Log.issue_id — FOREIGN KEY REFERENCES Issue(issue_id) ON DELETE CASCADE

Implementation:

5.1 DDL: Create Database and Tables

```
CREATE DATABASE IF NOT EXISTS civic_issues_db;
USE civic_issues_db;

CREATE TABLE IF NOT EXISTS Citizen (
  citizen_id INT AUTO_INCREMENT PRIMARY KEY,
  name       VARCHAR(255) NOT NULL,
  email      VARCHAR(255) UNIQUE NOT NULL,
  mobile     VARCHAR(20)  NOT NULL,
  area       VARCHAR(255) NOT NULL,
  password   VARCHAR(255) NOT NULL,
  role       ENUM('citizen','admin') DEFAULT 'citizen',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```

CREATE TABLE IF NOT EXISTS Issue (
  issue_id      INT AUTO_INCREMENT PRIMARY KEY,
  title         VARCHAR(255) NOT NULL,
  description   TEXT NOT NULL,
  area         VARCHAR(255) NOT NULL,
  category      ENUM('Roads','Water','Sanitation','Electricity','Parks','Others')
                NOT NULL DEFAULT 'Others',
  severity      ENUM('Low','Medium','High') NOT NULL,
  priority_score INT DEFAULT 0,
  status        ENUM('Open','In Progress','Resolved') DEFAULT 'Open',
  report_date   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  resolution_date TIMESTAMP NULL,
  citizen_id    INT NOT NULL,
  FOREIGN KEY (citizen_id) REFERENCES Citizen(citizen_id) ON DELETE CASCADE
);

```

```

CREATE TABLE IF NOT EXISTS Issue_Priority_Log (
  log_id        INT AUTO_INCREMENT PRIMARY KEY,
  issue_id      INT NOT NULL,
  old_priority  INT,
  new_priority  INT,
  change_date   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (issue_id) REFERENCES Issue(issue_id) ON DELETE CASCADE
);

```

5.2 Database Triggers

Four triggers automate priority scoring and audit logging:

Trigger 1: before_issue_insert — Auto-set priority_score on INSERT

```

DELIMITER //
CREATE TRIGGER before_issue_insert
BEFORE INSERT ON Issue
FOR EACH ROW
BEGIN
  IF NEW.severity = 'High' THEN SET NEW.priority_score = 90;
  ELSEIF NEW.severity = 'Medium' THEN SET NEW.priority_score = 60;
  ELSEIF NEW.severity = 'Low' THEN SET NEW.priority_score = 30;
  END IF;
END //
DELIMITER ;

```

Trigger 2: after_issue_insert — Log initial priority on INSERT

```
DELIMITER //
CREATE TRIGGER after_issue_insert
AFTER INSERT ON Issue
FOR EACH ROW
BEGIN
    INSERT INTO Issue_Priority_Log (issue_id, old_priority, new_priority)
    VALUES (NEW.issue_id, NULL, NEW.priority_score);
END //
DELIMITER ;
```

Trigger 3: before_issue_update — Recalculate score if severity changes

```
DELIMITER //
CREATE TRIGGER before_issue_update
BEFORE UPDATE ON Issue
FOR EACH ROW
BEGIN
    IF NEW.severity != OLD.severity THEN
        IF NEW.severity = 'High' THEN SET NEW.priority_score = 90;
        ELSEIF NEW.severity = 'Medium' THEN SET NEW.priority_score = 60;
        ELSEIF NEW.severity = 'Low' THEN SET NEW.priority_score = 30;
        END IF;
    END IF;
END //
DELIMITER ;
```

Trigger 4: after_issue_update — Log priority change on UPDATE

```
DELIMITER //
CREATE TRIGGER after_issue_update
AFTER UPDATE ON Issue
FOR EACH ROW
BEGIN
    IF NEW.priority_score != OLD.priority_score THEN
        INSERT INTO Issue_Priority_Log (issue_id, old_priority, new_priority)
        VALUES (NEW.issue_id, OLD.priority_score, NEW.priority_score);
    END IF;
END //
DELIMITER ;
```


5.3 DML: Sample Queries Used by the Application

INSERT — Register a Citizen

```
INSERT INTO Citizen (name, email, mobile, area, password, role)
VALUES ('Ramesh Patel', 'ramesh@gmail.com', '9876543210', 'Baner, Pune',
       '$2b$10$hashed_password_here', 'citizen');
```

INSERT — Create a New Issue (trigger fires automatically)

```
INSERT INTO Issue (title, description, area, category, severity, citizen_id)
VALUES ('Pothole on Main Road', 'Large pothole near Signal No. 4, causing
accidents',
       'Baner, Pune', 'Roads', 'High', 5);
-- TRIGGER fires: priority_score is automatically set to 90
```

SELECT — Get All Issues (Admin view, sorted by priority)

```
SELECT Issue.*, Citizen.name AS citizen_name
FROM Issue
JOIN Citizen ON Issue.citizen_id = Citizen.citizen_id
ORDER BY Issue.priority_score DESC, Issue.report_date ASC;
```

SELECT — Get Issues for a Specific Citizen

```
UPDATE Issue
SET status = 'Resolved',
    resolution_date = CASE WHEN 'Resolved' = 'Resolved'
                           THEN CURRENT_TIMESTAMP ELSE NULL END
WHERE issue_id = 12;
```

UPDATE — Change Issue Status to Resolved (Admin)

```
SELECT Issue.*, Feedback.rating AS user_rating
FROM Issue
LEFT JOIN Feedback ON Issue.issue_id = Feedback.issue_id
WHERE Issue.citizen_id = ?
ORDER BY Issue.report_date DESC;
```

UPDATE — Change Severity (Citizen), triggers recalculation

```
UPDATE Issue
SET severity = 'High'
WHERE issue_id = 7 AND citizen_id = 5;
-- TRIGGER fires: priority_score updated to 90, audit log entry created
```

SELECT — Public Statistics (Resolved/Active counts, avg response time)

```
SELECT COUNT(*) AS resolved_count FROM Issue WHERE status = 'Resolved';
SELECT COUNT(*) AS active_count FROM Issue WHERE status != 'Resolved';

SELECT AVG(TIMESTAMPDIFF(MINUTE, report_date, resolution_date)) AS avg_minutes
FROM Issue WHERE status = 'Resolved' AND resolution_date IS NOT NULL;

SELECT AVG(rating) AS avg_rating FROM Feedback;
```

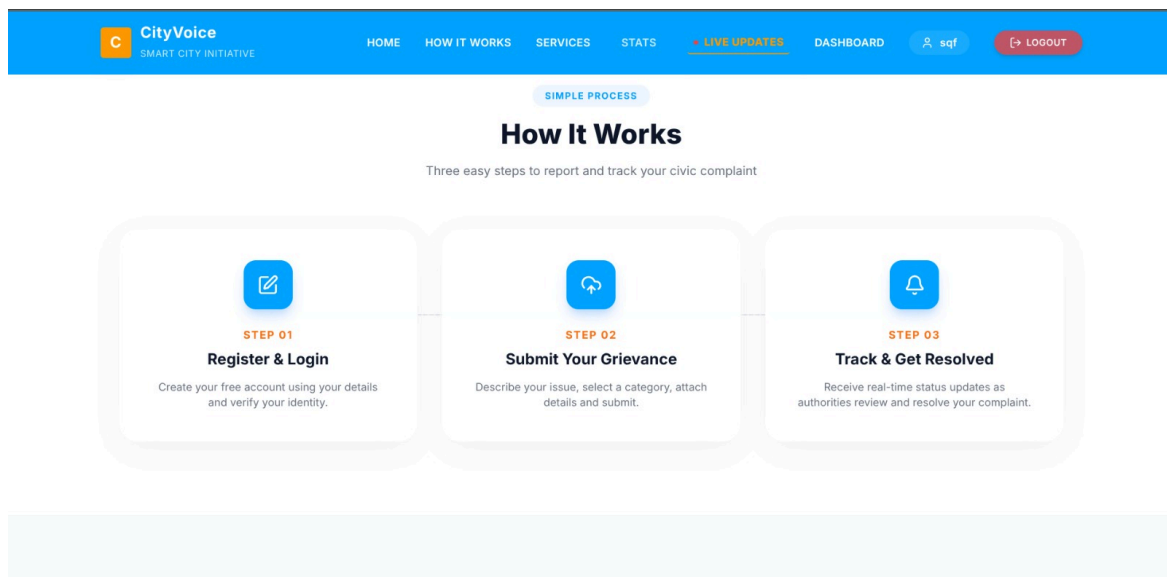
SELECT — Public Issues Feed (with optional category filter)

```
SELECT issue_id, title, description, area, category,
       severity, status, report_date, priority_score
FROM Issue
WHERE category = 'Roads' -- optional filter
ORDER BY report_date DESC LIMIT 50;
```

INSERT — Submit Citizen Feedback for Resolved Issue

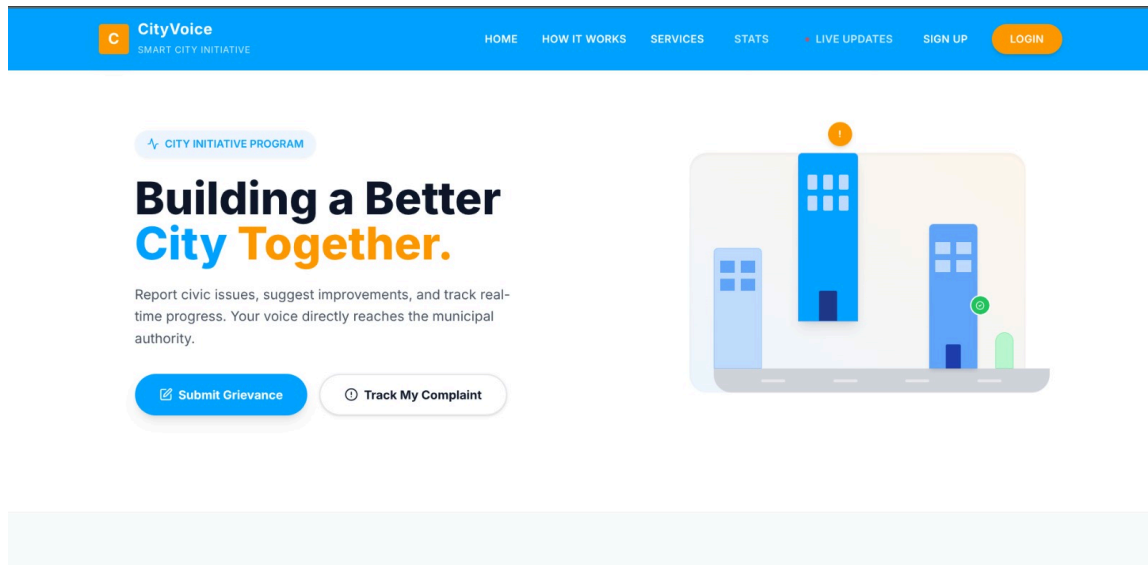
```
INSERT INTO Feedback (issue_id, citizen_id, rating, comment)
VALUES (12, 5, 4, 'Issue was fixed within 2 days. Good response!');
```

Results and Output:



The following section describes the four main functional views of the CityVoice application along with the database outcomes produced by each module.

Home Page- Public Live Feed & Statistics:



The landing page displays real-time statistics including total resolutions and average response times calculated via **TIMESTAMPDIFF**. Visitors can use a categorized grid to execute parameterized **SELECT queries** by department or view a live feed of the four most recently resolved issues.

Citizen Registration & Login:

The registration module hashes passwords with **encrypt (10 salt rounds)** before database insertion, while successful login returns a **30-day JWT token**. This token is stored in **localStorage** and automatically attached to all subsequent API requests as a **Bearer token** via an Axios interceptor.

Citizen Dashboard- Submit & Track Issues:

The screenshot shows the 'My Civic Issues' section of the CityVoice dashboard. It features a form titled 'File a new grievance' with the following fields: 'Issue Title' (containing 'Pipeline Leakage'), 'Category' (a dropdown menu with 'Roads' selected), 'Area / Location' (containing 'Pimpri'), and 'Severity Level' (a dropdown menu with 'High - Dangerous/Critical' selected). A 'Detailed Description' text area contains the text 'Pipeline has been leaking for past 2 days and causing severe traffic disruption.' A 'Submit Issue' button with a right arrow is at the bottom right of the form. The top navigation bar includes links for HOME, HOW IT WORKS, SERVICES, STATS, LIVE UPDATES, and DASHBOARD, along with a user profile icon labeled 'sqf' and a LOGOUT button.

The **Citizen Dashboard** uses a **LEFT JOIN** to display personal issue cards with real-time status and severity badges. Users can update severity to trigger database-level recalculations or submit 5-star ratings for resolved issues via the **Feedback** endpoint.

Admin Dashboard- Manage All Issues:

The screenshot shows the 'Admin Portal' section of the CityVoice dashboard. It features a search bar with a dropdown menu for 'Filters' (set to 'All Statuses') and a search input field labeled 'Search by Area...'. Below the search bar, a large white box with a green checkmark icon and the text 'Queue is empty' and 'No issues found matching your current filters.' is displayed. The top navigation bar includes links for DASHBOARD and System Admin, along with a user profile icon and a LOGOUT button.

The **Admin Dashboard** provides a prioritized, real-time filtered table of civic issues, using **JOIN queries** to link citizen and complaint data. Admins manage the lifecycle from **Open** to **Resolved**, while severity overrides automatically trigger **priority recalculations** and timestamps.

Conclusion:

The **Smart Grievance Handling System** effectively integrates a normalized **MySQL** database with a full-stack architecture to ensure high data integrity and automated priority scoring. By achieving **Third Normal Form (3NF)** and utilizing database triggers, the system automates complex logic and maintains tamper-proof audit logs. Key learning outcomes include the practical application of **DBMS concepts** such as schema normalization, aggregate functions for real-time statistics, and role-based access control. This resulted in a transparent and efficient platform suitable for municipal deployment. Looking forward, the project identifies scalable future enhancements like **Firestore Cloud Messaging** for notifications and **Google Maps API** for geographic density mapping. Overall, the project validates how automated backend logic can significantly improve civic issue resolution and institutional transparency.

References:

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
2. Connolly, T., & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management (6th ed.). Pearson.
3. MySQL 8.0 Reference Manual — Triggers. Oracle Corporation.
<https://dev.mysql.com/doc/refman/8.0/en/triggers.html>
4. Express.js Documentation. <https://expressjs.com/>
5. React Documentation — React.dev. <https://react.dev/>
6. JSON Web Tokens (JWT) — RFC 7519. <https://jwt.io/>
7. bcrypt — npm package documentation. <https://www.npmjs.com/package/bcrypt>
8. mysql2 — npm package documentation. <https://www.npmjs.com/package/mysql2>
9. Tailwind CSS Documentation. <https://tailwindcss.com/docs/>
10. Axios HTTP Client Documentation. <https://axios-http.com/docs/intro>